

LEVEL 4 — DEVELOPMENT TRACK FOUNDATIONS

Software Engineering Foundations

Disciplined practices for writing code that is correct, maintainable and safe to change

Covers: Clean code / SOLID principles · OOP / functional concepts · design patterns · type safety · error handling · logging · configuration · documentation

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
9. Software Engineering Foundations (Level 4)	3
9.1 Learning Position at a Glance	3
9.2 Clean Code & SOLID Principles	3
9.3 OOP & Functional Concepts.....	5
9.4 Design Patterns	5
9.5 Type Safety	6
9.6 Error Handling	7
9.7 Logging.....	7
9.8 Configuration.....	8
9.9 Documentation	8
9.10 Worked Exercise — Model Answer.....	9
9.11 Common Mistakes — Deeper Look.....	11
9.12 Advanced Extensions	12
9.13 Professional Applications	12

9. Software Engineering Foundations (Level 4)

Covers: Clean code/SOLID principles · OOP/functional concepts · design patterns · type safety · error handling · logging · configuration · documentation

9.1 Learning Position at a Glance

- **What it is:** Disciplined practices for writing code that is correct, maintainable and safe to change.
- **Why it matters:** Research code that 'just works once in a notebook' is a liability in production; a single silent bug in position-sizing logic can lose real money.
- **Prerequisites:** Python Fundamentals (Level 1).
- **Connects to Research:** Well-structured, typed, tested research code is dramatically faster to iterate on and far less prone to silent bugs than notebook-only code.
- **Connects to Development:** This is the foundation of the entire development track.
- **Connects to Trading:** Trading systems that mismanage errors or fail silently create real, unbounded financial risk.
- **Practical exercise:** Refactor a messy, function-only backtest script into a typed, documented set of classes (Signal, Portfolio, Backtester) with explicit error handling.
- **Common beginner mistakes:**
 - Writing 'research-only' code with no error handling, then deploying it unchanged to production.
 - Overusing inheritance/design patterns where a simple function would be clearer (over-engineering).
- **Advanced extensions:** Design patterns specific to trading systems (e.g. event-driven architectures, strategy pattern for pluggable signals).
- **Professional applications:** Any production or shared research codebase.

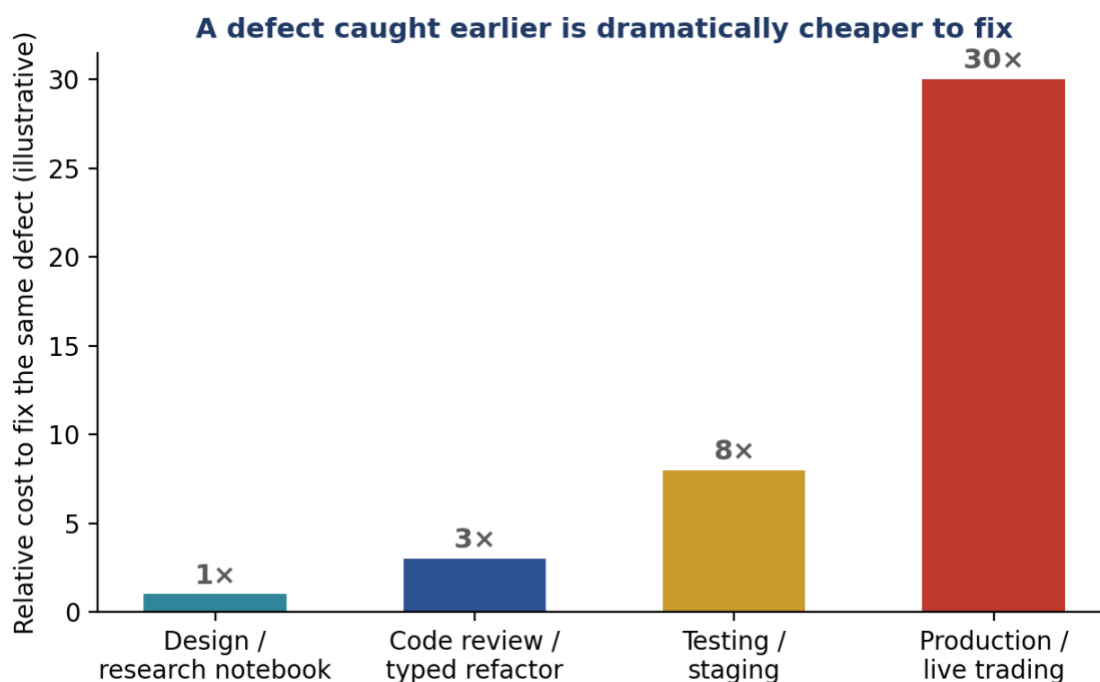


Figure 9.1 — The same defect is progressively more expensive to fix the later it is caught; the practices in this section (typing, tests, review) exist specifically to catch defects at the cheap end of this curve rather than the expensive end.

9.2 Clean Code & SOLID Principles

Clean code is code whose intent is obvious to a reader who did not write it — clear names, small functions that do one thing, and structure that mirrors the problem being solved rather than the order in which the author happened to think

of it. The SOLID principles are five specific, well-established design guidelines that, taken together, make object-oriented code easier to extend safely and harder to break accidentally.

The five SOLID principles of object-oriented design

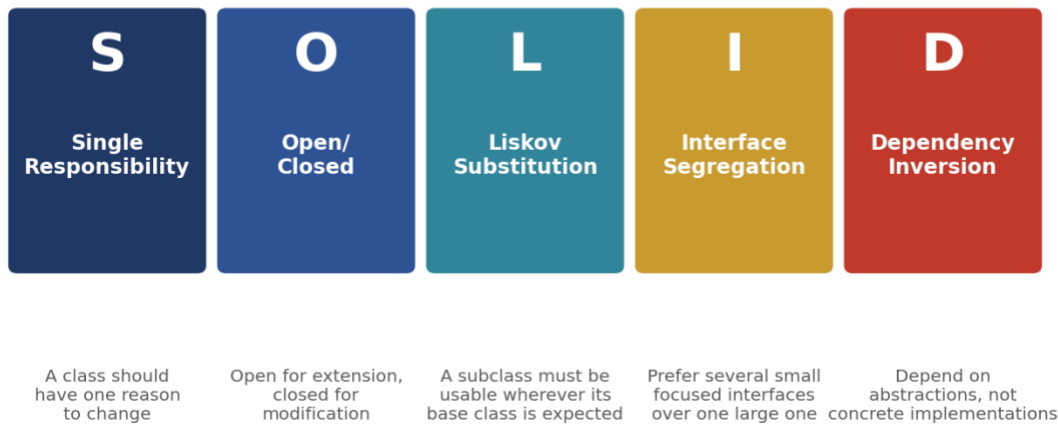


Figure 9.2 — The five SOLID principles. Each addresses a specific way that poorly structured object-oriented code becomes fragile as it grows.

Single Responsibility, illustrated

A class that both computes a trading signal and writes results to a database has two reasons to change: a change to the signal logic, or a change to the storage format. Splitting these into two classes means a change to one never risks breaking the other.

Example 9.1 — Single Responsibility Principle

```
# Before: one class doing two unrelated jobs
class MomentumSignal:
    def compute_and_save(self, prices, db_path):
        signal = prices.pct_change(20).rank(pct=True)
        signal.to_csv(db_path)  # storage concern mixed into signal logic
        return signal

# After: each class has exactly one reason to change
class MomentumSignal:
    def compute(self, prices):
        return prices.pct_change(20).rank(pct=True)

class SignalRepository:
    def save(self, signal, path):
        signal.to_csv(path)
```

Open/Closed, illustrated

Code is open for extension but closed for modification when a new behaviour can be added without editing existing, already-tested code — typically by adding a new subclass or function rather than adding another branch to an existing conditional.

Example 9.2 — Open/Closed Principle

```
# Before: adding a new signal type means editing this function again
def compute_signal(kind, prices):
    if kind == "momentum":
        return prices.pct_change(20)
    elif kind == "reversal":
        return -prices.pct_change(5)
    # every new signal type means another elif branch here
```

```
# After: new signal types are added, not edited in
class Signal:
    def generate(self, prices): raise NotImplementedError

class MomentumSignal(Signal):
    def generate(self, prices):
        return prices.pct_change(20)

class ReversalSignal(Signal):
    def generate(self, prices):
        return -prices.pct_change(5)
```

9.3 OOP & Functional Concepts

Object-oriented programming (OOP) groups data and the behaviour that operates on it together into classes, and is a natural fit when a system has meaningful, stateful entities — a Portfolio that holds positions and changes over time is a good OOP candidate. Functional programming favours pure functions — functions whose output depends only on their inputs, with no hidden state and no side effects — and is a natural fit for data transformations, such as computing a signal from a price series. Quant research code typically benefits from both: pure functions for calculations that should be easy to test and reason about in isolation, and classes for the small number of genuinely stateful entities (portfolios, live connections, running backtests) that need to track state across many calls.

Example 9.3 — A pure function alongside a genuinely stateful class

```
# A pure function: same input always gives the same output,
# with no hidden state and nothing else changed as a side effect.
def sharpe_ratio(returns, periods_per_year=252):
    return returns.mean() / returns.std() * (periods_per_year ** 0.5)

# A stateful class: genuinely needs to track state across many calls.
class Portfolio:
    def __init__(self, starting_cash: float):
        self.cash = starting_cash
        self.positions: dict[str, float] = {}

    def update(self, ticker: str, quantity: float, price: float) -> None:
        self.positions[ticker] = self.positions.get(ticker, 0) + quantity
        self.cash -= quantity * price
```

- **Prefer a pure function when:** The task is a calculation with no meaningful state to track between calls — computing a signal, a risk metric, or a transformation of a data series.
- **Prefer a class when:** The entity genuinely accumulates state over time that later calls depend on — a portfolio's positions, a live connection's session, a running backtest's progress.

9.4 Design Patterns

A design pattern is a named, reusable solution to a recurring design problem — a shared vocabulary that lets engineers communicate a whole design idea ('just use a Strategy pattern here') in a few words rather than re-explaining the structure from scratch each time. A handful of patterns recur constantly in quant research and trading systems.

Pattern	Problem it solves	Typical use in a quant system
Strategy	Swap out an algorithm's behaviour without changing the code that uses it	Pluggable trading signals: a Backtester holds a Signal interface and can run any concrete signal that implements it

Pattern	Problem it solves	Typical use in a quant system
Factory	Decouple the code that creates an object from the code that uses it	Constructing the correct data-loader or broker-connector class based on a configuration value
Observer	Let one or more objects react automatically when another object's state changes	A risk dashboard and a logger both reacting to every new fill, without the execution code knowing either exists
Decorator	Add behaviour to an object without modifying its class	Wrapping a data-fetch function with retry logic or caching without touching the original function
Singleton	Ensure only one instance of a class exists application-wide	A single shared configuration object or database connection pool (use sparingly — see §9.11)

The Strategy pattern, illustrated

Example 9.4 — Strategy pattern: Backtester is closed for modification, open to any Signal

```
from typing import Protocol

class Signal(Protocol):
    def generate(self, prices) -> "pd.Series": ...

class Backtester:
    def __init__(self, signal: Signal):
        self.signal = signal          # depends on the interface, not a concrete class

    def run(self, prices):
        weights = self.signal.generate(prices)
        return self._simulate(weights, prices)

# Any new signal can be run without changing Backtester at all:
backtester = Backtester(signal=MomentumSignal())
results = backtester.run(prices)
```

9.5 Type Safety

Python does not require type annotations, but adding them turns a whole class of bugs — passing a price where a return was expected, or a list where a single value was expected — from a silent runtime failure (or worse, a silent wrong answer) into an error a type checker (such as mypy) or an IDE catches before the code ever runs.

Example 9.5 — Adding type hints makes a function's contract explicit

```
# Before: no types. This function's expectations are guesswork.
def position_size(capital, price, risk_pct):
    return capital * risk_pct / price

# After: explicit types make the contract unambiguous, and a type
# checker will flag a call site that passes the wrong kind of value.
def position_size(capital: float, price: float, risk_pct: float) -> float:
    if price <= 0:
        raise ValueError(f"price must be positive, got {price}")
    return capital * risk_pct / price
```

Type hints are most valuable exactly at the boundaries the practical exercise targets: the interfaces between Signal, Portfolio and Backtester, where a mismatch (a DataFrame passed where a Series was expected) is otherwise a silent source of subtly wrong backtest results rather than a loud, immediate error.

9.6 Error Handling

Research code can often get away with letting an error crash the notebook — the researcher sees the traceback and fixes it. Production trading code cannot: an unhandled error partway through a trading loop can leave positions, orders or state in an inconsistent condition with real money at stake. Deliberate error handling means deciding, for every operation that can fail, what should happen when it does — not simply hoping it will not.

Example 9.6 — Turning a silent failure mode into an explicit, specific exception

```
# Before: research-only code with no error handling at all.
def get_price(ticker):
    return data_feed.fetch(ticker)["last_price"]

# After: explicit, specific error handling with a clear failure mode.
class DataUnavailableError(Exception):
    """Raised when a required price cannot be retrieved."""

def get_price(ticker: str) -> float:
    try:
        payload = data_feed.fetch(ticker)
    except ConnectionError as exc:
        raise DataUnavailableError(
            f"Could not reach data feed for {ticker}") from exc
    if "last_price" not in payload:
        raise DataUnavailableError(f"No price field for {ticker}")
    return payload["last_price"]
```

- **Fail loudly and specifically:** Raise a specific, well-named exception rather than letting a generic error (or worse, a silently wrong value like zero or None) propagate.
- **Decide the failure mode deliberately:** For every external call (a data feed, a broker API), decide explicitly whether a failure should retry, fall back to a default, or halt the system — rather than leaving the answer to whatever the underlying library happens to do.
- **Never silently swallow an exception:** A bare except: pass block hides exactly the information needed to diagnose why a strategy produced an unexpected result.

9.7 Logging

print() statements disappear when a script closes and give no way to distinguish routine information from a serious warning. A logging framework records timestamped, severity-tagged messages that persist to a file, can be filtered by severity in production, and provide the audit trail needed to reconstruct exactly what a trading system did and why — often the only way to diagnose an incident after the fact.

Example 9.7 — Structured, severity-tagged logging in place of print()

```
import logging

logger = logging.getLogger("backtester")

def run_backtest(signal, prices):
    logger.info("Starting backtest: %d price rows, signal=%s",
                len(prices), type(signal).__name__)
    try:
        weights = signal.generate(prices)
    except Exception:
        logger.exception("Signal generation failed")
        raise
    if weights.isna().any():
        logger.warning("Signal produced %d NaN weights", weights.isna().sum())
    logger.info("Backtest complete")
    return weights
```

- **DEBUG / INFO:** Routine detail useful during development, or high-level milestones useful for a normal operational record.
- **WARNING:** Something unexpected happened but the system can continue — worth a human eventually noticing.
- **ERROR / CRITICAL:** An operation failed, or the system cannot safely continue — worth alerting a human immediately.

9.8 Configuration

Hardcoding values that will plausibly need to change — a risk limit, a data path, an API endpoint — directly into logic couples configuration to code: changing an environment (research vs. staging vs. production) requires editing and redeploying code rather than changing a setting. Externalising configuration keeps the two concerns separate.

Example 9.8 — Named configuration in place of unexplained magic numbers

```
# Before: magic numbers and paths hardcoded directly into logic.
def size_position(capital):
    return capital * 0.02 / 50000    # what do 0.02 and 50000 mean here?

# After: named, typed, centrally defined configuration.
from dataclasses import dataclass

@dataclass(frozen=True)
class RiskConfig:
    max_risk_pct: float = 0.02
    max_position_notional: float = 50_000.0

def size_position(capital: float, config: RiskConfig) -> float:
    return capital * config.max_risk_pct / config.max_position_notional
```

A typed configuration object (as opposed to a loose dictionary read from a file with no validation) gets the same benefit described in §9.5: a typo in a configuration key, or a value of the wrong type, is caught immediately rather than surfacing as a mysterious wrong answer deep inside a backtest.

9.9 Documentation

Documentation is not an afterthought bolted onto finished code; it is part of the interface a class or function presents to everyone else who will use it, including the original author six months later. A docstring should state what a function does, what its parameters and return value mean, and any behaviour a caller needs to know about — particularly what happens when something goes wrong.

Example 9.9 — A docstring that documents the edge case, not just the happy path

```
def sharpe_ratio(returns: "pd.Series", periods_per_year: int = 252) -> float:
    """Compute the annualised Sharpe ratio of a return series.

    Args:
        returns: Periodic (e.g. daily) simple returns, not prices.
        periods_per_year: Number of return periods per year, used to
            annualise the ratio (252 for daily, 12 for monthly).

    Returns:
        The annualised Sharpe ratio. Returns float('nan') if the
        input series has zero variance.
    """
    if returns.std() == 0:
        return float("nan")
    return returns.mean() / returns.std() * (periods_per_year ** 0.5)
```

Documentation type	Audience	What it should contain
Docstring	Anyone calling this specific function or class	Purpose, parameters, return value, and any non-obvious edge-case behaviour
README	Someone new to the whole codebase	What the project does, how to install and run it, and where to start reading
Architecture notes	Engineers making structural changes	Why the system is structured the way it is, and what trade-offs were made
Inline comments	A reader of this specific block of code	Why the code does something non-obvious — not a restatement of what it does

9.10 Worked Exercise — Model Answer

Exercise: Refactor a messy, function-only backtest script into a typed, documented set of classes (Signal, Portfolio, Backtester) with explicit error handling.

The messy starting point

Before

```
# messy_backtest.py – everything in one script, no types, no error handling
import pandas as pd

def backtest(prices, lookback, cash):
    signal = prices.pct_change(lookback).rank(pct=True) - 0.5
    positions = {}
    for date in prices.index:
        w = signal.loc[date]
        for ticker in prices.columns:
            positions[ticker] = w[ticker] * cash
    # no handling of missing data, no validation, no logging,
    # no way to swap in a different signal without editing this function
    return positions
```

The refactored design

Refactored design: Signal, Portfolio, Backtester and ExecutionSimulator as collaborating, single-responsibility classes

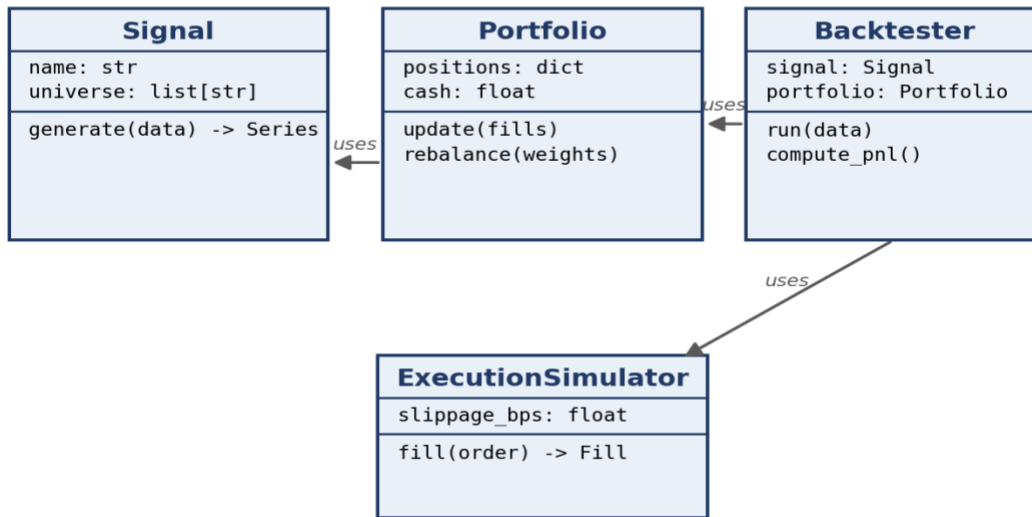


Figure 9.3 — The refactored backtest as four collaborating, single-responsibility classes: Signal, Portfolio, Backtester and ExecutionSimulator, matching the Strategy-pattern structure introduced in Example 9.4.

After

```

import logging
from dataclasses import dataclass, field
from typing import Protocol
import pandas as pd

logger = logging.getLogger("backtester")

class Signal(Protocol):
    def generate(self, prices: pd.DataFrame) -> pd.DataFrame:
        """Return target weights indexed the same way as prices."""
        ...

class MomentumSignal:
    def __init__(self, lookback: int = 20):
        self.lookback = lookback

    def generate(self, prices: pd.DataFrame) -> pd.DataFrame:
        return prices.pct_change(self.lookback).rank(pct=True, axis=1) - 0.5

@dataclass
class Portfolio:
    cash: float
    positions: dict = field(default_factory=dict)

    def rebalance(self, weights: pd.Series, prices: pd.Series) -> None:
        for ticker, w in weights.items():
            price = prices.get(ticker)
            if price is None or price <= 0:
                logger.warning("Skipping %s: invalid price", ticker)
                continue
            self.positions[ticker] = w * self.cash / price
    
```

```

class BacktestError(Exception):
    """Raised when a backtest cannot be run to completion."""

class Backtester:
    def __init__(self, signal: Signal, portfolio: Portfolio):
        self.signal = signal
        self.portfolio = portfolio

    def run(self, prices: pd.DataFrame) -> Portfolio:
        if prices.empty:
            raise BacktestError("Cannot backtest an empty price history")
        weights = self.signal.generate(prices)
        for date in weights.index:
            self.portfolio.rebalance(weights.loc[date], prices.loc[date])
        logger.info("Backtest complete over %d dates", len(weights))
        return self.portfolio

# Usage: swapping the signal never requires touching Backtester itself.
backtester = Backtester(signal=MomentumSignal(lookback=20),
                        portfolio=Portfolio(cash=1_000_000.0))
final_portfolio = backtester.run(prices)

```

Every practice covered in this section shows up in the refactor: Signal, Portfolio and Backtester each have one responsibility (§9.2); Signal is a Protocol so new signals can be added without editing Backtester (§9.2, §9.4); every public method is typed (§9.5); invalid prices and empty inputs are handled explicitly rather than silently (§9.6); progress and problems are logged, not printed (§9.7); and the lookback and starting cash are named, explicit parameters rather than buried magic numbers (§9.8).

9.11 Common Mistakes — Deeper Look

Mistake: Writing ‘research-only’ code with no error handling, then deploying it unchanged to production

Research code is written under an implicit assumption — the data is clean, the researcher is watching, and a crash simply means trying again — that becomes false the moment the same code runs unattended against live data feeds and real capital. The messy_backtest.py example in §9.10 works perfectly well for a single, supervised research run, but deployed unchanged as a live trading loop it would silently mis-handle any missing price, with no record of what happened or why. The fix is not to write research code more defensively from the very first exploratory line — that would slow down legitimate, fast iteration — but to treat the transition from notebook to production as a deliberate hardening step, per Figure 9.1, rather than skipping it under time pressure.

Mistake: Overusing inheritance/design patterns where a simple function would be clearer (over-engineering)

The patterns in §9.4 solve specific, recurring problems; applied where the underlying problem does not exist, they add indirection and ceremony without any corresponding benefit. A one-off calculation used in a single place does not need a Strategy pattern, an abstract base class and a factory to construct it — a plain function is clearer, easier to test, and easier for the next reader to follow in one pass.

⚠ Case study: a Signal hierarchy nobody needed

- A team built an elaborate five-level class hierarchy (BaseSignal → StatisticalSignal → CrossSectionalSignal → RankedSignal → MomentumSignal) for a single momentum calculation used in exactly one place.
- No second signal was ever added to justify the hierarchy, and new team members consistently spent their first day just tracing which of the five classes actually contained the one line of real logic.
- The team eventually collapsed the hierarchy back to a single MomentumSignal class implementing the Signal protocol from Example 9.4 — exactly as much structure as the Strategy pattern needs, and no more.

9.12 Advanced Extensions

Design patterns specific to trading systems: as a codebase moves from research-style backtesting toward live or near-live trading, an event-driven architecture becomes increasingly natural — rather than a single script executing top to bottom, the system becomes a set of independent components that each react to events (a new price tick, a fill confirmation, a risk breach) arriving on a queue.

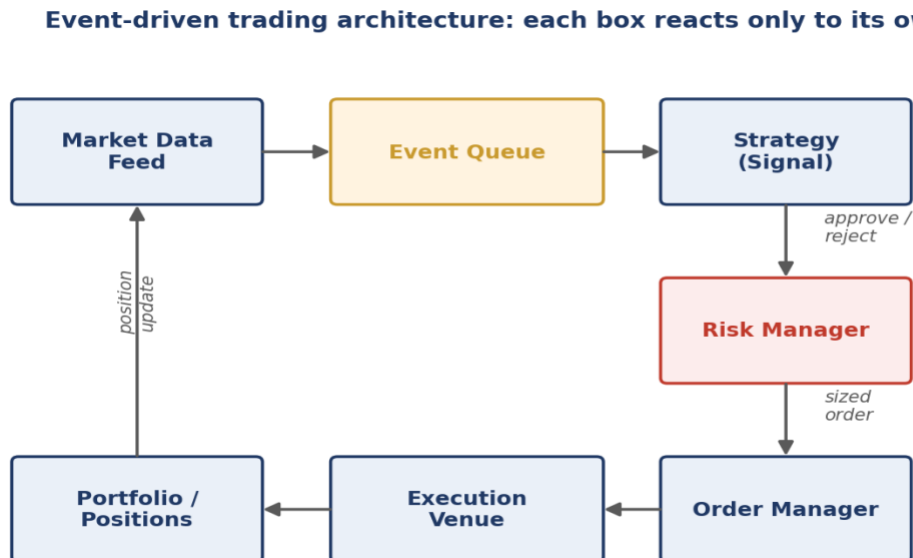


Figure 9.4 — An event-driven architecture built from the same single-responsibility discipline as §9.2: each component reacts only to events on its own queue, and a new component (e.g. an additional risk check) can be inserted without modifying the others — the Open/Closed Principle applied at the system level.

- **Event-driven architecture:** Decouples components in time as well as in code: the Strategy component does not call the Order Manager directly, it emits an event that the Order Manager (and anything else interested, per the Observer pattern in §9.4) can react to independently.
- **Strategy pattern for pluggable signals:** The same Strategy pattern introduced in Example 9.4 for backtesting extends naturally to live trading: a live system can swap, combine, or A/B test multiple Signal implementations behind the identical Backtester/live-runner interface.

Depth beyond this should be driven by a specific system being built, not general interest — the right amount of architectural sophistication is whatever the current system's actual scale and reliability requirements justify, per the over-engineering caution in §9.11.

9.13 Professional Applications

- **Any production or shared research codebase:** Every practice in this section — SOLID structure, typing, explicit error handling, logging, externalised configuration, and real documentation — applies wherever more than one person will read or maintain the code, or wherever it will run without a researcher watching over it in real time; this is the default baseline for professional quant code, not an optional upgrade reserved for a final production release.